

Smart Expense Tracker

Relational Database Design, 3NF Normalization Proofs, Architecture & Technical Walkthrough

TARGET PLATFORM	BACKEND ENGINE	DATABASE ARCHITECTURE
Apache + MySQL (XAMPP for Windows)	PHP 8 (Strict Types & Native PDO)	MySQL 8.0+ (InnoDB, 3NF Normalized)
CLIENT INTERFACE		
Modern Fintech UI • Chart.js 4.4		

1. Executive Summary

The **Smart Expense Tracker** is a full-stack personal finance application designed to combine **academic DBMS rigor** with a **modern fintech user experience**. It provides end-to-end income and expense management, periodic budget variance tracking, milestone savings goals, automated financial alerts, and an educational catalog demonstrating **20 fundamental relational DBMS queries**.

The backend is architected in **pure PHP 8 and MySQL** without introducing external runtime dependencies (such as Node.js or Python). Security is strictly enforced using **100% prepared SQL statements**, Bcrypt password hashing, session regeneration, and CSRF token protection.

2. Technology Stack & Tool Justification

Every tool and library in this project was selected for academic defensibility and zero-configuration local deployment:

Technology / Tool	Role in Application	Why We Specifically Chose It
PHP 8.0+	Server-side business logic & HTML template compilation	Native execution on Apache, zero compilation step, cross-platform compatibility, and <code>declare(strict_types=1)</code> for strict type safety.
MySQL 8 (InnoDB)	Relational Database Management System (RDBMS)	ACID compliance, referential integrity via Foreign Keys (<code>ON DELETE CASCADE / RESTRICT</code>), B-Tree performance indexing, and virtual Relational Views.

Technology / Tool	Role in Application	Why We Specifically Chose It
PHP Data Objects (PDO)	Database connectivity abstraction layer	Configured with <code>PDO::ATTR_EMULATE_PREPARES => false</code> , ensuring MySQL compiles SQL statements natively and completely eliminates SQL Injection vulnerabilities.
Bcrypt (PASSWORD_BCRYPT)	Cryptographic password hashing	Industry-standard salted one-way hashing with an adaptive work factor that protects against brute-force and rainbow-table attacks (unlike MD5/SHA1).
Chart.js 4.4	Client-side financial data visualization	Lightweight canvas charting engine that renders responsive line charts, grouped cashflow bars, and category doughnut charts without Node.js dependencies.
XAMPP Environment	Local Apache + MySQL deployment stack	Standard evaluation environment in university DBMS laboratories. Bundles Apache and MySQL into an executable control panel for Windows.

3. System Workflow & Request Lifecycle

When a user interacts with the application, the request flows through a 4-tier pipeline:

The 4-Step Processing Cycle:

- Client Dispatch:** The browser issues an HTTP request (GET or POST) to Apache on port 80.
- Security & Session Gate:** PHP loads `includes/auth.php`, initializes `session_start()`, verifies user authentication (`require_login()`), and checks CSRF tokens on write operations.
- Database Execution:** The script connects to MySQL via `getConnection()` in `config/database.php` and executes prepared SQL queries with bound parameter arrays.
- Hydration & Client Render:** PHP compiles financial metrics into the HTML DOM and serializes datasets into `window.DashboardData` for Chart.js interactive rendering.

4. Complete Walkthrough of All 25 PHP Files

The codebase is organized into five logical modules:

Module A: Core Configuration & Security Guards

File	Execution Responsibility & Logic
config/database.php	Establishes a singleton PDO connection to MySQL. Enforces strict UTF-8 (utf8mb4), exception error modes, and displays a recovery troubleshooting UI if the MySQL service is offline.
includes/auth.php	The core security module. Provides require_login() session guards, CSRF token generation/verification, currency/date formatters, and the 5-rule get_smart_insights() engine.
includes/header.php	HTML document head template. Injects responsive viewports, Google Fonts (Plus Jakarta Sans), FontAwesome vector icons, and the global design system stylesheet.
includes/navbar.php	Sticky top navigation bar. Queries live account Net Balance in real time and provides the universal + Add Transaction modal trigger.
includes/sidebar.php	Responsive navigation sidebar and mobile slide-out drawer with active route tracking and sign-out confirmation modal triggers.
includes/footer.php	Global application footer. Contains the universal Quick Add Transaction Modal, the custom Confirmation Dialog Modal (replacing browser confirm()), and loads Chart.js 4.4.

Module B: Entry, Authentication & Session Flow

File	Execution Responsibility & Logic
index.php	Application entry point. Checks if the user is already authenticated (redirecting to dashboard) or serves the public presentation page.
login.php	Authenticates users. Validates credentials via password_verify() , regenerates session IDs against fixation attacks, and includes a 1-click Demo Account filler.
register.php	User account provisioning. Validates input bounds, verifies email uniqueness, hashes passwords via PASSWORD_BCRYPT , seeds a welcome alert, and establishes the user session.
logout.php	Clears \$_SESSION , expires session cookies, destroys the session, and redirects to login with an informational notice.

Module C: Financial Ledger Operations

File	Execution Responsibility & Logic
<code>dashboard.php</code>	The central financial dashboard. Computes Top KPIs (Income, Expense, Balance, Savings), compiles 6-month trends for area and bar charts, doughnut category allocations, and renders budget threshold bars.
<code>transactions.php</code>	The complete financial ledger. Implements multi-criteria parameterized filtering (search keyword, category, type, date range, sort order) and pagination fallbacks.
<code>add_transaction.php</code>	Records new financial transactions. If an expense is entered, automatically evaluates whether it causes a budget overrun and inserts alert warnings into <code>alerts</code> .
<code>edit_transaction.php</code>	Updates existing ledger tuples. Enforces user ownership in the WHERE clause (<code>transaction_id = ? AND user_id = ?</code>) to eliminate IDOR security vulnerabilities.
<code>delete_transaction.php</code>	Safe record removal. Validates CSRF tokens and user ownership before executing a parameter-bound DELETE query.

Module D: Budgets, Categories, Savings & Reports

File	Execution Responsibility & Logic
<code>budgets.php</code>	Establishes category budget thresholds. Runs a <code>LEFT JOIN</code> query to evaluate actual spent amounts against limits and displays color-coded progress bars (Green <80%, Amber ≥80%, Red ≥100%).
<code>categories.php</code>	Manages income and expense classification categories. Calculates aggregate volume per category and manages user-custom categories.
<code>savings.php</code>	Manages milestone savings reserves. Computes target completion percentages and supports atomic fund deposits via SQL addition.
<code>analytics.php</code>	Deep spending analytics. Computes highest spending categories, 14-day daily velocity area charts, and uses SQL subqueries to compute average monthly expenditure.
<code>reports.php</code>	Multi-statement reporting generator (Monthly, Category, Income, Budget, Savings). Supports one-click CSV export and a print-optimized layout.
<code>queries.php</code>	Academic DBMS catalog. Houses and dynamically executes all 20 essential SQL queries with live output tables, copy buttons, and normalization documentation.

File	Execution Responsibility & Logic
profile.php	Account administration. Displays live DBMS account telemetry and handles secure password changes with old-password verification.

Module E: Asynchronous REST APIs

File	Execution Responsibility & Logic
api/dashboard_data.php	Authenticated JSON endpoint returning live summary metrics (income, expenses, balance, monthly expense, category breakdowns).
api/analytics_data.php	Authenticated JSON endpoint returning 12-month aggregated time-series cashflow arrays for client graphs.
api/transaction_data.php	Authenticated JSON endpoint returning paginated transaction records bounded between 1 and 100 entries.

 5. Relational Database Normalization Proofs (3NF)

The database schema (`expense_tracker.sql`) consists of 6 tables and 3 relational views, designed in accordance with **Third Normal Form (3NF)**:

Normal Form	DBMS Requirement	Implementation Proof in This Project
First Normal Form (1NF)	Attributes must be atomic; no multi-valued attributes; explicit Primary Key defined.	All columns hold indivisible scalar data. Financial figures use <code>DECIMAL(10,2)</code> (not comma-separated strings). Each table defines an explicit surrogate primary key (e.g., <code>user_id INT AUTO_INCREMENT PRIMARY KEY</code>).
Second Normal Form (2NF)	Must satisfy 1NF; no partial dependencies (all non-key attributes fully dependent on candidate keys).	All relations use single-attribute surrogate primary keys. Non-key attributes (such as <code>amount</code> or <code>description</code>) depend entirely on the entire primary key, eliminating partial dependencies.
Third Normal Form (3NF)	Must satisfy 2NF; no transitive functional dependencies (non-key attributes must not depend on other non-key attributes).	In <code>transactions</code> , we store <code>category_id</code> (foreign key) but do not store <code>category_name</code> or <code>icon</code> . Category attributes are resolved via <code>JOIN</code> operations, eliminating update anomalies.

Relational Views

- 1. `v_monthly_financial_summary` : Aggregates total income, expense, and net savings per user per month.
- 2. `v_budget_status` : Dynamically computes budget utilization percentage and status (`ON_TRACK` , `WARNING` , `EXCEEDED`).
- 3. `v_category_spending` : Computes category-wise counts, sums, averages, and extremes.

6. Catalog of 20 Essential Relational SQL Queries

The application demonstrates all 20 required DBMS operations in `queries.php` :

#	Relational DBMS Concept	Application Usage in Project
1	Basic SELECT (Projection)	Profile attribute retrieval in <code>profile.php</code>
2	INSERT (DML)	Financial transaction entry in <code>add_transaction.php</code>
3	UPDATE (Mutation)	Record modification with user ID check in <code>edit_transaction.php</code>
4	DELETE (Referential Removal)	Safe record removal in <code>delete_transaction.php</code>
5	WHERE Clause (Selection)	30-day date boundary filtering in <code>transactions.php</code>
6	ORDER BY (Sorting)	Multi-attribute sorting (amount DESC, date DESC)
7	GROUP BY (Aggregation)	Cashflow aggregation grouped by transaction type
8	HAVING Clause (Group Filter)	Filtering high-volume categories where total spent > \$100
9	SUM Function	Lifetime income and expense summation in <code>dashboard.php</code>
10	AVG Function	Calculating average expenditure per transaction
11	COUNT Function	Category transaction frequency counting
12	MAX Function	Finding the single highest expenditure logged
13	MIN Function	Finding the lowest expenditure logged
14	INNER JOIN	Joining transactions with categories for badge colors and icons
15	LEFT JOIN	Category portfolio share calculation including unspent categories

#	Relational DBMS Concept	Application Usage in Project
16	Multiple-Table JOIN	Simultaneously joining transactions, users, and categories
17	Subquery (Correlated/Nested)	Finding transactions exceeding the user's historical average
18	Monthly Expense Analysis	Year-Month trajectory analysis via <code>DATE_FORMAT()</code>
19	Category Allocation Breakdown	Computing category percentage share of total spending
20	Budget Exceeded Analysis	Calculating budget overruns and utilization percentages

? 7. Viva Voce Defense Quick Reference

Q1: Why is `DECIMAL(10,2)` used instead of `FLOAT` or `DOUBLE`?

Answer: Floating-point numbers use binary approximations which produce inexact rounding errors (e.g. $0.1 + 0.2 = 0.30000000000000004$). In financial systems, every cent must balance perfectly, so MySQL's fixed-point `DECIMAL(10,2)` data type is strictly required.

Q2: How does your application prevent SQL Injection?

Answer: We use PHP Data Objects (PDO) with native prepared statements (`PDO::ATTR_EMULATE_PREPARES ⇒ false`). User data is never concatenated into SQL strings; it is transmitted to the database engine separately through parameter placeholders (`?`), making SQL injection impossible.

Q3: How do you enforce referential integrity?

Answer: We use the InnoDB engine with foreign key constraints. When a user is deleted, `ON DELETE CASCADE` cleans up related transactions and budgets. For categories with recorded transactions, `ON DELETE RESTRICT` prevents orphaned records and preserves historical audit trails.

Q4: Why use views instead of repeated complex queries?

Answer: Views (like `v_budget_status`) encapsulate complex multi-table joins and calculations into virtual tables. This promotes code reusability, modularity, and allows the MySQL query optimizer to optimize execution plans.